

Kafka + Redis a gyakorlatban

Event-driven rendszerek Kubernetesen

K8s (k3s) · Helm · GitOps (ArgoCD) · Prometheus

Oktatási anyag a Posta DevOps csapatának

A „Posta Csomagkövető” demóprojekt alapján

github.com/mattycska/Kafka · 2026. augusztus

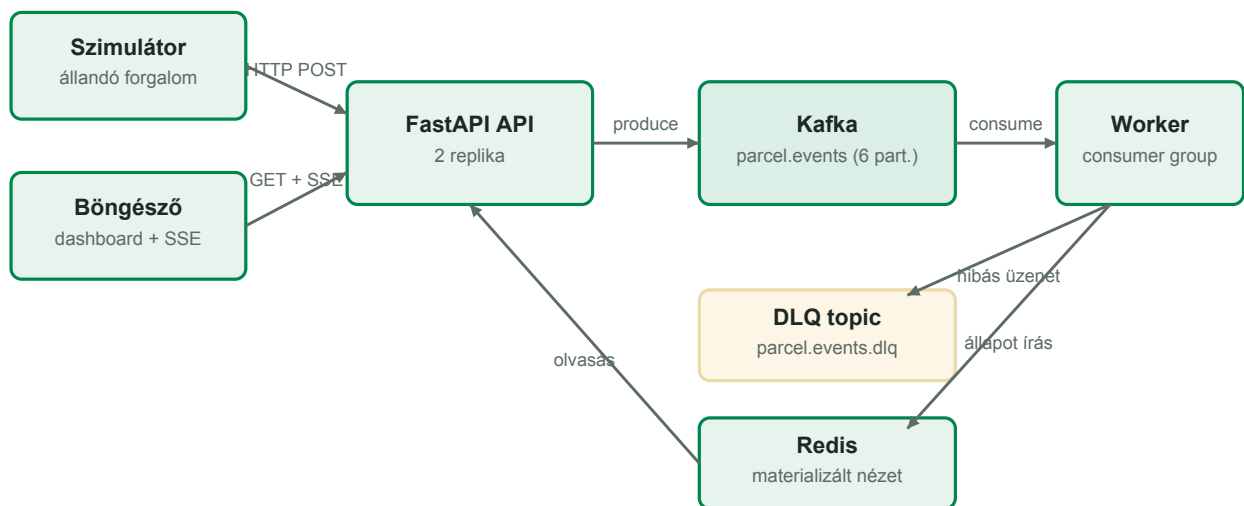
Tartalomjegyzék

Tartalomjegyzék	2
1. Mit építettünk és miért?	4
1.1. Az eseményvezérelt gondolkodás	4
1.2. CQRS: az írási és olvasási út szétválasztása	4
1.3. A repó térképe	5
2. Kafka-alapok a demón keresztül	6
2.1. Topic, partíció, offset	6
2.2. Kulcs szerinti particionálás = sorrendgarancia	6
2.3. Producer-garanciák: acks és idempotencia	6
2.4. Consumer group és rebalance	6
2.5. Kézbetítési szemantikák — a legfontosabb téma	7
2.6. Dead Letter Queue (DLQ)	7
2.7. KRaft: Kafka ZooKeeper nélkül	7
3. Redis: a gyors olvasási oldal	8
3.1. A demó Redis-kulcsai	8
3.2. Materializált nézet: a Redis mint származtatott állapot	8
3.3. Cache-aside minta	8
3.4. TTL: az adat élettartama	8
3.5. Pub/Sub: élő közvetítés	9
4. Az alkalmazás anatómiája	10
4.1. Az állapotgép: a domain logika szíve	10
4.2. Az API írási útja	10
4.3. A worker feldolgozási ciklusa	10
4.4. Graceful shutdown: a K8s és az alkalmazás kézfogása	10
4.5. A szimulátor: a demó motorja	11
5. Konténerizálás: egy image, három szerep	12
5.1. Multi-stage build	12
5.2. Lokális fejlesztés: docker compose	12
6. Kubernetes (k3s) a demóban	13
6.1. Deployment vs. StatefulSet — mikor melyik?	13
6.2. Liveness vs. readiness — nem ugyanaz!	13
6.3. Erőforrás-kérések és limitek	13

6.4. Ingress + TLS	13
6.5. Konfiguráció: ConfigMap + envFrom	13
7. Helm: a Kubernetes csomagkezelője	14
7.1. A chart felépítése	14
7.2. Values: egy chart, sok környezet	14
7.3. Helperek és névképzés	14
7.4. Hookok: műveletek a telepítés életciklusában	14
7.5. A napi munka parancsai	15
8. GitOps és ArgoCD: a Git az igazság forrása	16
8.1. App-of-apps: a klaszter tartalomjegyzéke	16
8.2. A teljes CI/CD-kör — PAT nélkül	16
8.3. Mit látsz az ArgoCD felületén?	17
9. Monitorozás: Prometheus + Grafana	18
9.1. A metrika-típusok — a demó példáival	18
9.2. Hogyan találja meg a Prometheus az alkalmazást?	18
9.3. Grafana dashboard mint kód	18
9.4. Mit érdemes nézni a demó dashboardján?	18
10. Hands-on gyakorlatok	19
10.1. Ismerkedés: kövess végig egy csomagot!	19
10.2. Consumer group skálázás és rebalance	19
10.3. DLQ: küldj poison pillt!	19
10.4. Idempotencia bizonyítása	19
10.5. A materializált nézet újraépítése (replay)	20
10.6. Cache-aside megfigyelése	20
10.7. Self-healing és GitOps-drift	20
10.8. Változtatás GitOps-módra	20
10.9. Grafana: olvasd a rendszert!	20
11. Merre tovább? A demótól az éles rendszerig	22
12. Függelék: parancspuska	23
Kafka CLI (a broker podban futtatva)	23
Redis CLI	23
Kubernetes / Helm / ArgoCD	23
Az API gyorstesztje	23

1. Mit építettünk és miért?

Ez a tananyag egyetlen, végigkövethető demóprojektre épül: a **Posta Csomagkövetőre**. A rendszer csomagok életútját követi feladástól kézbesítésig — de a valódi cél nem a csomagkövetés, hanem az, hogy minden modern DevOps-alapfogalmat egy kézzelfogható, működő rendszeren keresztül tanuljunk meg: eseményvezérelt architektúra, Kafka, Redis, konténerizálás, Kubernetes, Helm, GitOps és monitorozás.



1. ábra — A demórendszer architektúrája

1.1. Az eseményvezérelt gondolkodás

A hagyományos (kérés-válasz) rendszerben a webalkalmazás közvetlenül írja az adatbázist, és a hívó megvárja az eredményt. Az eseményvezérelt rendszerben ehelyett **eseményeket** rögzítünk: „a csomagot felvették”, „a csomag megérkezett a szegedi feldolgozóba”. Az esemény megtörtént tény — nem lehet visszavonni, legfeljebb újabb eseménnyel korrigálni.

- **Lazán csatolt komponensek:** a termelő (producer) nem tudja és nem is érdekli, ki dolgozza fel az eseményt — új fogyasztó bekötése nem igényel változtatást a termelőn.
- **Terheléselnyelés:** a Kafka pufferként viselkedik; ha a fogyasztó lassabb, az események várnak, nem vesznek el.
- **Visszajátszhatóság:** az eseménynapló megmarad — a származtatott állapot (nálunk a Redis) bármikor újraépíthető.

1.2. CQRS: az írási és olvasási út szétválasztása

A demó API-ja tudatosan aszimmetrikus. Az **írási oldal** (csomagfeladás, életút-esemény) csak annyit tesz: validálja a kérés formáját, és beküldi az eseményt a Kafkába — majd azonnal `202 Accepted` választ ad. Az **olvasási oldal** (csomag lekérdezés, statisztika) kizárólag a Redisből olvas. A kettő között a **worker** teremt kapcsolatot: a Kafka eseményeit feldolgozva előállítja a Redis „materializált nézetet”.

TIPP — Ha közvetlenül feladás után kérdezed le a csomagot, előfordulhat, hogy pár ezredmásodpercig még 404-et kapsz — ez az **eventual consistency** (végül konzisztens rendszer). Nem hiba, hanem tudatos architektúrális döntés, amiért cserébe a rendszer skálázható és hibatűrő.

1.3. A repó térképe

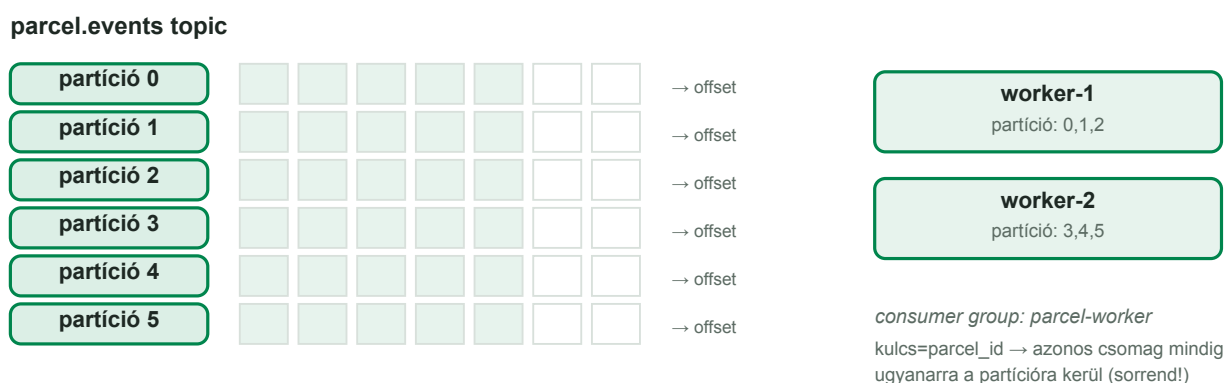
Útvonal	Tartalom
src/parcel_tracker/	Python forrás: API, worker, szimulátor, közös modulok
tests/	Pytest tesztek (Kafka/Redis nélkül futnak: fake-ekkel)
Dockerfile	Egy image, három szerep (api / worker / simulator)
docker-compose.yml	Lokális fejlesztői stack egy paranccsal
helm/parcel-tracker/	Helm chart: Kafka, Redis, app, monitoring
.github/workflows/ci.yml	CI/CD: teszt → build → ghcr → GitOps visszaírás

2. Kafka-alapok a demón keresztül

Az Apache Kafka elosztott, naplóalapú üzenetplatform. A legfontosabb mentális modell: a Kafka nem „postaláda”, ahonnan az üzenet kivétel után eltűnik, hanem **hozzáfűzhető napló** (append-only log), amelyből minden fogyasztó a saját pozíciója (offset) szerint olvas.

2.1. Topic, partíció, offset

A **topic** logikai csatorna (nálunk: `parcel.events`). Minden topic **partíciókra** oszlik — a partíció a párhuzamosság és a sorrend egysége. Az üzenetek a partíción belül szigorú sorrendben állnak, és minden üzenetnek van egy sorszáma, az **offset**.



2. ábra — 6 partíció, 2 worker: minden partíciónak pontosan egy gazdája van a csoporton belül

A demó topicja 6 partícióval jött létre. Ez azt jelenti: a `parcel-worker` consumer groupban **legfeljebb 6 worker** dolgozhat párhuzamosan — a hetedik már nem kapna partíciót, tétlenül várna. A partíciószám tehát kapacitástervezési döntés, és utólag csak felfelé módosítható.

2.2. Kulcs szerinti particionálás = sorrendgarancia

Melyik partícióra kerül egy üzenet? Ha adunk neki **kulcsot**, a Kafka a kulcs hash-e alapján dönt — azonos kulcs mindig azonos partíció. A demóban a kulcs a `parcel_id`: így egy adott csomag eseményei garantáltan sorrendben dolgozódnak fel, miközben a különböző csomagok párhuzamosan futnak. Ez a minta a Kafka-tervezés egyik legfontosabb fogása.

```
await producer.send_and_wait(  
    topic,  
    value=event.to_kafka_bytes(),  
    key=event.parcel_id.encode("utf-8"), # kulcs = sorrend a csomagon belül  
)
```

kafka_client.py — a kulcs megválasztása

2.3. Producer-garanciák: acks és idempotencia

- `acks=all` — a broker csak akkor igazolja vissza az üzenetet, ha minden szinkron replika megkapta. Egy brokeres demónál nincs látható különbség, élesben ez az adatvesztés elleni alapvédelem.
- `enable_idempotence=True` — újraküldésnél (hálózati hiba, timeout) a broker felismeri és eldobja a duplikátumot, így a producer oldal nem tud kettőzni.

2.4. Consumer group és rebalance

A fogyasztók **consumer group**ba szerveződnek: a Kafka a topic partícióit szétosztja a csoport tagjai között, és minden partíciót pontosan egy tag birtokol. Ha új worker lép be (skálázás!) vagy egy meglévő kiesik, a Kafka **rebalance**-t hajt végre: újraosztja a partíciókat. A demó workere ezt logolja is — a skálázási gyakorlatban élőben nézzük majd.

```
kubectl -n kafka-demo scale deploy kafka-demo-worker --replicas=4
kubectl -n kafka-demo logs -l app=kafka-demo-worker -f | grep Rebalance
```

Rebalance megfigyelése skálázás közben

2.5. Kézbesítési szemantikák — a legfontosabb téma

Szemantika	Mit jelent?	Mikor fordul elő?
at-most-once	Az üzenet legfeljebb egyszer dolgozódik fel; el is veszhet.	Ha a fogyasztó a feldolgozás ELŐTT commitolja az offsetet.
at-least-once	Az üzenet biztosan feldolgozódik, de duplikáció lehetséges.	Ha a feldolgozás UTÁN commitolunk — a demó ezt teszi.
exactly-once	Pontosan egyszer — drága, és csak határok között értelmezhető.	Kafka tranzakciók + idempotens feldolgozás kombinációja.

A demó workere **at-least-once** módban fut (`enable_auto_commit=False`, kézi commit a feldolgozás után), és a duplikátumokat a Redis **idempotencia-kulccsal** szűri ki: minden esemény `event_id`-jára egy `SET NX` „foglalást” teszünk. Ha a kulcs már létezik, az esemény duplikátum, és eldobjuk. A kettő együtt a gyakorlatban `exactly-once` viselkedést ad — ezt hívjuk **effectively-once**-nak.

FIGYELEM — A tökéletes `exactly-once` elosztott rendszerben általában nem éri meg az árát. A bevált recept: `at-least-once` szállítás + idempotens feldolgozás. Ezt kérdezd vissza minden integrációnál: „mi történik, ha kétszer kapom meg ugyanazt az üzenetet?”

2.6. Dead Letter Queue (DLQ)

Mi történjen, ha egy üzenet feldolgozhatatlan — rossz a formátuma, vagy az állapotgép szerint értelmezhetetlen (pl. KEZBESITVE esemény egy még fel sem dolgozott csomagra)? Ha a worker ilyenkor hibával leáll, az üzenet **poison pill**-ként végtelen ciklusban blokkolná a partíciót. A demó ehelyett a hibás üzenetet a `parcel.events.dlq` topicra teszi, az eredeti tartalommal és a hiba okával együtt, majd halad tovább.

```
{"reason": "ervénytelen_allapotatmenet",
 "detail": "Érvénytelen átmenet: FELVEVE állapotból nem fogadható el KEZBESITVE...",
 "original": "{...az eredeti üzenet...}",
 "failed_at": "2026-08-12T19:34:52.796206+00:00"}
```

Egy valódi DLQ-bejegyzés a demóból

2.7. KRaft: Kafka ZooKeeper nélkül

A régebbi Kafka-verziók külön ZooKeeper klasztert igényeltek a metaadat-kezeléshez. A modern Kafka (3.3+) **KRaft** módban fut: a metaadat-konszenzust maga a Kafka intézi Raft protokollal. A demóban egy node látja el a broker és a controller szerepet is — oktatáshoz ideális, élesben 3+ broker és elkülönített controller-kvórum a minimum.

3. Redis: a gyors olvasási oldal

A Redis memóriában tartott kulcs-érték adatbázis — de a „kulcs-érték” megnevezéstől szerény leírás: az értékek gazdag adatszerkezetek lehetnek, és pont ez adja a Redis erejét. A demó szándékosan sokféle szerkezetet használ, mindegyiket más célra.

3.1. A demó Redis-kulcsai

Kulcs	Típus	Szerep
parcel:{id}	HASH	A csomag aktuális állapota (státusz, hely, súly...)
parcel:{id}:history	LIST	Esemény-idővonal, RPush-sal bővül
dedup:{event_id}	STRING	Idempotencia-kulcs: SET NX + TTL
stats:status_counts	HASH	Csomagszám státuszonként (HINCRBY)
stats:center:events	ZSET	Feldolgozóközpont-toplista (ZINCRBY)
stats:delivered:{nap}	STRING	Napi kézbesítés-számláló, 40 nap TTL
cache:stats:overview	STRING	Cache-aside: előre kiszámolt összesítő
events:live	PUB/SUB	Élő esemény-közvetítés a dashboardnak

3.2. Materializált nézet: a Redis mint származtatott állapot

Kulcsgondolat: a demóban a **Redis tartalma bármikor eldobható**. A forrás igazsága (source of truth) a Kafka eseménynapló — a Redis csak egy olvasásra optimalizált vetület. Ha a Redis kiürül, a consumer group offsetjének visszaállításával a worker az összes eseményt újrátölti, és a nézet újraépül. Ezt a gyakorlatok között ki is próbáljuk.

TIPP — Ezért nincs a demó Redisének perzisztenciája (sem RDB, sem AOF): tudatos döntés, amit a chart kommentje dokumentál is. Élesben a Redis szerepétől függ: cache-nek nem kell, elsődleges tárnak igen.

3.3. Cache-aside minta

Az összesítő statisztika (/api/stats/overview) több Redis-hívást igényel. Ezer egyidejű dashboard-felhasználónál ez fölösleges terhelés lenne — ezért az eredményt 5 másodpercre cache-eljük:

```
cached = await self._redis.get("cache:stats:overview")
if cached:
    return (**json.loads(cached), "cache": "hit") # gyors út
# ... drága aggregáció ...
await self._redis.set("cache:stats:overview", json.dumps(result), ex=5)
```

redis_store.py — cache-aside rövid TTL-lel

A minta neve onnan jön, hogy a cache az adatút **mellett** (aside) áll: először a cache-t nézzük; ha nincs találat (miss), kiszámoljuk az értéket, és visszatesszük. A dashboard jobb felső kártyáján élőben látszik a HIT/MISS váltakozás.

3.4. TTL: az adat élettartama

Memória-adatbázisnál a legfontosabb kérdés: **mi jár le és mikor?** A demóban minden növekvő adathalmaznak van lejárata: a csomagadatok 3 nap, a dedup-kulcsok 1 nap, a napi számlálók 40 nap után tűnnek el. A Redis `maxmemory` limitje `noeviction` szabállyal fut: ha betelne, az írás hibát dob — jobb hangosan elhasalni, mint csendben adatot veszíteni.

3.5. Pub/Sub: élő közvetítés

A worker minden feldolgozott eseményt publikál az `events:live` csatornára. Az API erre iratkozik fel, és **Server-Sent Events** (SSE) folyamként továbbítja a böngészőnek — ettől él a dashboard. A Pub/Sub „tűz és felejt” jellegű: aki épp nem figyel, az lemarad az üzenetről. Nálunk ez pont jó (élő feed); garantált kézbesítéshez Kafka vagy Redis Streams való.

4. Az alkalmazás anatómiája

4.1. Az állapotgép: a domain logika szíve

A csomag életútját szigorú állapotgép írja le: minden státuszról csak meghatározott események fogadhatók el. Ez a `models.py`-ban egyetlen táblázat — az érthető, tesztelhető üzleti logika iskolapéldája:

```
ALLOWED_TRANSITIONS = {
    None: {FELVETEL},
    FELVEVE: {FELDOLGOZAS},
    FELDOLGOZAS_ALATT: {FELDOLGOZAS, SZALLITAS},
    SZALLITAS_ALATT: {FELDOLGOZAS, KEZBESITES_INDITVA},
    KEZBESITES_ALATT: {KEZBESITVE, SIKERTELEN_KEZBESITES},
    SIKERTELEN_KEZBESITES: {KEZBESITES_INDITVA, VISSZAKULDES},
    KEZBESITVE: set(), # végállapot
    VISSZAKULDVE: set(), # végállapot
}
```

models.py — az állapotgép mint adat

Figyeld meg, hol fut az ellenőrzés: **a workerben, nem az API-ban**. Az API bármilyen (formailag helyes) eseményt befogad — ha az átmenet érvénytelen, azt a worker deríti ki, és DLQ-ba teszi. Elosztott rendszerben az írási végpont nem láthatja megbízhatóan az aktuális állapotot (közben más események is sorban állhatnak), ezért a döntés ott születik, ahol az állapot ténylegesen előáll.

4.2. Az API írási útja

```
@router.post("/parcels", status_code=202)
async def create_parcel(body: CreateParcelRequest, request: Request):
    event = ParcelEvent(parcel_id=new_parcel_id(),
                        event_type=EventType.FELVETEL, ...)
    await send_event(_producer(request), settings.kafka_topic, event)
    return {"parcel_id": event.parcel_id, "status": "elfogadva", ...}
```

routes.py — az író végpont csak Kafkába termel

A 202 `Accepted` státuszkód azt üzeni a hívónak: „megkaptam és elfogadtam a kérést, de a feldolgozása még folyamatban van”. Ez az aszinkron API-k szabványos válasza.

4.3. A worker feldolgozási ciklusa

- **1. Parse + validálás:** a nyers bytes → `ParcelEvent` (pydantic). Hibás formátum → DLQ, `validacios_hiba` okkal.
- **2. Idempotencia:** SET dedup:{event_id} NX EX 86400 — ha már létezik, duplikátum, eldobjuk.
- **3. Állapotgép:** érvénytelen átmenet → DLQ, `ervenytelen_allapotatmenet` okkal.
- **4. Redis-írás:** egyetlen pipeline-ban (tranzakcióban) frissül a csomag hash, a history, az összes statisztika, és megy ki a Pub/Sub üzenet.
- **5. Offset commit:** kötegenként, a feldolgozás UTÁN — ez adja az at-least-once garanciát.

4.4. Graceful shutdown: a K8s és az alkalmazás kézfogása

Amikor a Kubernetes le akar állítani egy podot (skálázás, új verzió), először `SIGTERM` jelzést küld, és csak a türelmi idő (`terminationGracePeriodSeconds`, nálunk 30 mp) után öl. A worker a `SIGTERM`-re befejezi az aktuális köteget, commitolja az offsetet, lezárja a Kafka- és Redis-kapcsolatokat — így az

újraindulás üzenetvesztés és fölösleges duplikáció nélkül zajlik.

```
for sig in (signal.SIGTERM, signal.SIGINT):
    loop.add_signal_handler(sig, stop_event.set)
...
while not stop_event.is_set():
    batch = await consumer.getmany(timeout_ms=1000, max_records=100)
    ...feldolgozás...
    await consumer.commit()
```

worker/main.py — rendezett leállítás

4.5. A szimulátor: a demó motorja

A szimulátor nem a Kafkába ír közvetlenül, hanem a **publikus HTTP API-t** hívja — így a teljes út folyamatosan terhelődik, és a rendszer „magától él”. Percenként ~10 csomagot ad fel, végigviszi őket az életúton (2 műszakos feldolgozóközpont-útvonalakkal, 10% sikertelen kézbesítéssel), és a csomagok ~3%-ánál **szándékosan hibás** eseménysorrendet küld, hogy a DLQ soha ne legyen üres. Élesben ugyanez a minta szintetikus monitorozásként (synthetic monitoring) köszön vissza.

5. Konténerizálás: egy image, három szerep

5.1. Multi-stage build

```
# ---- Build réteg: függőségek + csomag telepítése ----
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml README.md ./
COPY src ./src
RUN pip install --no-cache-dir --prefix=/install .

# ---- Futtató réteg: minimális, nem-root ----
FROM python:3.12-slim
COPY --from=builder /install /usr/local
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
ENTRYPOINT ["python", "-m"]
CMD ["parcel_tracker.api"]
```

Dockerfile — a lényeg

- **Multi-stage:** a build szerszámai (pip cache, source) nem kerülnek a végső image-be — kisebb és biztonságosabb.
- **Nem-root futás:** a `USER 10001` numerikus uid-t a Kubernetes `runAsNonRoot` szabálya ellenőrizni is tudja.
- **ENTRYPOINT + CMD minta:** az `entrypoint ["python", "-m"]`, az argumentum dönti el a szerepet — a Helm chartban a `worker args: ["parcel_tracker.worker"]`-rel indul. Egy image-et buildelünk, verziózunk és skálázunk, három szerepben.

5.2. Lokális fejlesztés: docker compose

A `docker compose up --build` egyetlen paranccsal elindítja a teljes stacket: Kafka (KRaft), topic-inicializáló, Redis, API, worker, szimulátor. A compose fájl két tanulságos részlete:

- **Függőségi sorrend:** `depends_on + condition: service_healthy / service_completed_successfully` — az app csak akkor indul, ha a Kafka egészséges és a topicok létrejöttek.
- **Két listener:** a konténerhálózaton `kafka:9092`, a hostról `localhost:29092` — a Kafka „advertised listeners” koncepciója már itt, lokálisan megjelenik.

TIPP — A worker lokális skálázása: `docker compose up --scale worker=3` — a rebalance a logokban azonnal látszik. Ugyanazt a jelenséget látod, mint K8s-en a replikaszám-változtatásnál.

6. Kubernetes (k3s) a demóban

A k3s a Kubernetes könnyűsúlyú, egy binárisba csomagolt disztribúciója — kis szerverekre, edge környezetekre és tanulásra ideális, de teljes értékű K8s API-t ad. A demó minden erőforrása egy dedikált `kafka-demo` namespace-ben él.

6.1. Deployment vs. StatefulSet — mikor melyik?

Szempont	Deployment (API, worker, Redis)	StatefulSet (Kafka)
Pod-identitás	csereszabatos, véletlen név	stabil sorszám: <code>kafka-0</code>
Hálózati név	csak a Service közös neve	saját, stabil DNS név podonként
Tároló	jellemzően nincs / közös	podhoz kötött PVC (<code>volumeClaimTemplates</code>)
Mikor?	állapotmentes szolgáltatások	adatbázisok, brokerek

A Kafka-nak azért kell StatefulSet, mert a broker a saját **hirdetett címét** (`advertised listener`) mondja el a klienseknek — ehhez stabil DNS név kell (`kafka-demo-kafka-0.kafka-demo-kafka-headless...`), amit a **headless Service** ad. A kliensek a normál ClusterIP Service-en (`kafka-demo-kafka:9092`) bootstrapelnek, majd a brokertől kapott stabil címen folytatják.

6.2. Liveness vs. readiness — nem ugyanaz!

- **Liveness** (`/healthz`): „él-e a processz?” Ha nem, a kubelet **újraindítja** a konténert. Szándékosan buta: mindig 200, ha a processz válaszképes.
- **Readiness** (`/readyz`): „fogadhat-e forgalmat?” Ha nem, a pod **kikerül a Service mögül**, de nem indul újra. A demó API-ja itt a Redis-kapcsolatot is ellenőrzi.

FIGYELEM — Gyakori hiba a kettő összemosása: ha a liveness próbába függőségeket (adatbázis-ping) teszel, egy Redis-kimaradás az ÖSSZES API podot újraindítatja — az újraindítás pedig nem hozza vissza a Redist. Függőség-ellenőrzés a readinessbe való.

6.3. Erőforrás-kérések és limitek

Minden konténer deklarálja, mennyi CPU-t/memóriát **kér** (`requests` — ütemezési ígéret) és mennyit **kaphat legfeljebb** (`limits`). A demó szándékosan takarékos: a teljes stack elfér ~1,5 GB memóriában. A Kafka JVM-heap-jét külön is szabályozzuk (`KAFKA_HEAP_OPTS`), mert a JVM nem a K8s limitből számol.

6.4. Ingress + TLS

Az `ingress-nginx` a klaszter kapuja: hosztnév alapján irányítja a forgalmat a Service-ekhez. A `cert-manager` a `letsencrypt-prod` ClusterIssuerrel automatikusan szerez és újít TLS-tanúsítványt a `kafka-demo.mattlab.hu` címre — a chartban ez egyetlen annotáció.

6.5. Konfiguráció: ConfigMap + envFrom

Az alkalmazás minden beállítása környezeti változó (12-factor elv). A Helm chart egy ConfigMapbe rendereli őket, a podok `envFrom`-mal húzzák be. Így ugyanaz az image fut lokálisan és a klaszterben — csak a környezet más.

7. Helm: a Kubernetes csomagkezelője

Nyers YAML-lel egy többkomponensű alkalmazás gyorsan kezelhetetlenné válik: ismétlődő címkék, környezetenként eltérő értékek, összehangolt frissítés. A Helm **chart**ba csomagolja a manifeszteteket: sablonok (templates) + állítható értékek (values) + verziózott egység.

7.1. A chart felépítése

```
helm/parcel-tracker/
├─ Chart.yaml           # név, verzió, leírás
├─ values.yaml          # az állítható „gombok” – a CI ezt frissíti!
└─ templates/
    ├── _helpers.tpl     # névképzés, közös címkék
    ├── configmap.yaml   # app-konfiguráció (env)
    ├── kafka-statefulset.yaml + kafka-services.yaml
    ├── redis.yaml
    ├── api.yaml / worker.yaml / simulator.yaml
    ├── ingress.yaml
    ├── topics-job.yaml  # Helm hook: topic-létrehozás
    ├── servicemonitors.yaml
    └─ grafana-dashboard.yaml
```

A chart térképe

7.2. Values: egy chart, sok környezet

A sablonok a `{{ .Values.* }}` hivatkozásokon keresztül paraméterezhetők: replikaszám, image tag, erőforrások, a szimulátor sebessége, a monitoring ki-be kapcsolása. Ugyanaz a chart más values-szal más környezetet ad — teszt/éles szétválasztás egyetlen fájlnyi különbséggel.

```
# részlet a values.yaml-ból
image:
  repository: ghcr.io/mattycska/kafka-demo
  tag: "sha-1a2b3c4"      # <- a CI írja, az ArgoCD innen szinkronizál
worker:
  replicas: 2
simulator:
  enabled: true
parcelsPerMinute: 10
```

values.yaml — az állítható felület

7.3. Helperek és névképzés

A `_helpers.tpl` adja a következetes neveket: a release névből (nálunk `kafka-demo`) képződik minden erőforrásnév — `kafka-demo-api`, `kafka-demo-kafka` stb. Ez azért fontos, mert a komponensek **egymásra DNS-névvel hivatkoznak** (a ConfigMapben:

`KAFKA_BOOTSTRAP_SERVERS=kafka-demo-kafka:9092`) — a névképzésnek kiszámíthatónak kell lennie.

7.4. Hookok: műveletek a telepítés életciklusában

A topicokat nem a broker hozza létre automatikusan (az `auto.create` ki van kapcsolva — a partíciószám tudatos döntés!), hanem egy **hook Job** a telepítés/frissítés után:

```
annotations:
  helm.sh/hook: post-install,post-upgrade
  helm.sh/hook-delete-policy: before-hook-creation,hook-succeeded
```

topics-job.yaml — Helm hook annotációk

ArgoCD alatt a Helm hookok ArgoCD hookokká fordulnak (a post-install PostSync lesz): a Job a sikeres sync után fut le, majd törlődik.

7.5. A napi munka parancsai

```
helm lint helm/parcel-tracker          # statikus ellenőrzés
helm template kafka-demo helm/parcel-tracker | less  # mit renderel?
helm upgrade --install kafka-demo helm/parcel-tracker \
  --namespace kafka-demo --create-namespace          # kézi telepítés
helm get values kafka-demo -n kafka-demo # mi van épp kint?
```

Helm-parancspuska

TIPP — GitOps környezetben a `helm upgrade`-et szinte soha nem futtatod kézzel — az ArgoCD teszi, a Gitben lévő chart alapján. A `helm template` viszont a mindennapi hibakeresés eszköze marad.

8. GitOps és ArgoCD: a Git az igazság forrása

A GitOps alapelve egy mondatban: **a klaszter kívánt állapota Gitben él, és egy operátor (nálunk az ArgoCD) folyamatosan odaigazítja a valóságot**. Nem mi nyomjuk be a változást a klaszterbe (push), hanem a klaszter húzza be a Gitből (pull).

- **Auditálhatóság:** minden változás egy commit — ki, mikor, mit, miért.
- **Visszaállíthatóság:** a rollback egy `git revert`.
- **Nincs kézi kubectl:** a klaszterhez írási jog szinte senkinek nem kell — a támadási felület és az emberi hiba is zsugorodik.
- **Önjavítás:** ha valaki kézzel belenyúl a klaszterbe, az ArgoCD visszaállítja a Gitben deklarált állapotot (selfHeal).

8.1. App-of-apps: a klaszter tartalomjegyzéke

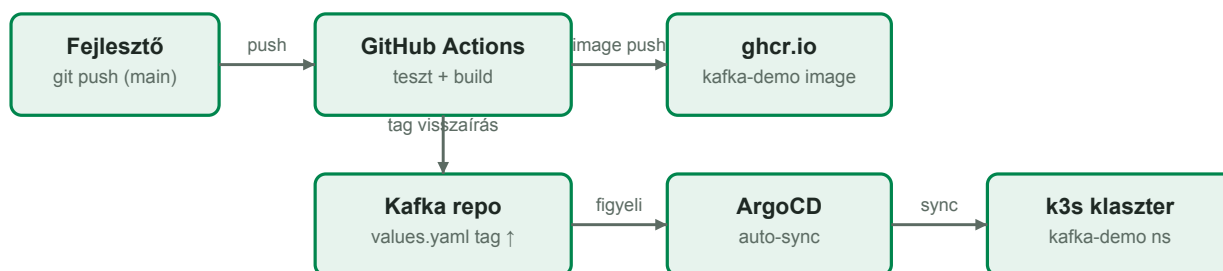
A gitops repóban egy **root Application** mutat az `apps/` mappára, amelyben minden alkalmazásnak egy-egy Application manifesztje van. Új alkalmazás felvétele = egy YAML fájl commitolása. A mi appunk:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kafka-demo
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/mattycska/Kafka.git # az APP repója!
    targetRevision: main
    path: helm/parcel-tracker # Helm chart útvonal
  destination:
    server: https://kubernetes.default.svc
    namespace: kafka-demo
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [ CreateNamespace=true ]
```

apps/kafka-demo.yaml a gitops repóban

Figyeld meg: a forrás **nem a gitops repo**, hanem az alkalmazás saját (publikus) repója. Multi-repo GitOps: a kód, a chart és a CI egy helyen él és együtt verziózódik; a gitops repo csak a „mi fusson a klaszteren” katalógusa.

8.2. A teljes CI/CD-kör — PAT nélkül



3. ábra — A push-tól a futó podig: minden lépés automatikus

A kör zárása a CI utolsó lépése: a workflow a frissen buildelt image **commit-SHA alapú tagjét** visszaírja a `values.yaml`-ba, és commitolja. Az ArgoCD ezt a commitot látja meg, és kiteszi az új verziót. Nem kell hozzá Personal Access Token, mert:

- a GitHub Actions beépített `GITHUB_TOKEN`-je a **saját repójába** tud commitolni (`contents: write`) és a **ghcr.io-ra** tud publikálni (`packages: write`);
- az ArgoCD **publikus** repókat hitelesítés nélkül olvas;
- a k3s **publikus** ghcr image-et pull secret nélkül húz.

FIGYELEM — Miért commit-SHA a tag, és miért nem `latest`? Mert a `latest` **mozgó cél**: nem látszik belőle, mi fut éppen, nem lehet rá visszaállni, és a cache-elés is kiszámíthatatlan. Az immutábilis tag a GitOps alapkövetelménye.

8.3. Mit látsz az ArgoCD felületén?

- **Sync status**: a Git és a klaszter egyezik-e (Synced / OutOfSync).
- **Health**: az erőforrások tényleges állapota (Healthy / Progressing / Degraded).
- **Erőforrásfa**: az Application alatt minden Deployment, Pod, Service, Ingress — kattintva logok és események.
- **History**: melyik commit mikor került ki — és egy kattintás a rollback.

9. Monitorozás: Prometheus + Grafana

9.1. A metrika-típusok — a demó példáival

Típus	Mit mér?	Példa a demóból
Counter	csak nő; sebességét a rate() adja	parcel_events_produced_total
Gauge	pillanatnyi érték, le-föl mozog	parcel_status_count{status=...}
Histogram	eloszlás vödrökben; percentilis számolható	parcel_event_processing_seconds

Arany szabály: a countert sosem ábrázoljuk nyersen (monoton nő), hanem `rate(...[ $__rate_interval ])` formában — ez adja az esemény/másodperc sebességet.
Percentilishez: `histogram_quantile(0.95, sum by (le) (rate(..._bucket[...])))`.

9.2. Hogyan találja meg a Prometheus az alkalmazást?

A kube-prometheus-stack **ServiceMonitor** erőforrásokkal dolgozik: a ServiceMonitor kimondja, mely Service-ek mely portján, milyen útvonalon és milyen gyakran kell kaparni (scrape). A demóban az API-nak és a workernek külön ServiceMonitora van.

FIGYELEM — Klasszikus csapda: a Prometheus csak a `release: kube-prometheus-stack` labelű ServiceMonitorokat veszi fel. Ha lemarad a label, minden „működik”, csak épp adat nincs — némán. Ha nincs metrika, ez legyen az első, amit ellenőrzöl.

9.3. Grafana dashboard mint kód

A Grafana sidecar minden `grafana_dashboard: "1"` labelű ConfigMapból automatikusan betölti a dashboardot. A demó chartja így szállítja a sajátját: a deploy után a Grafanában magától megjelenik a „Kafka Demo — Posta Csomagkövető” dashboard — kattintgatás, kézi import nincs. A dashboard is kód, verziózva, review-zva.

9.4. Mit érdemes nézni a demó dashboardján?

- **Termelt vs. feldolgozott események:** a két görbének együtt kell mozognia — ha szétválnak, a worker lemaradt (lag).
- **DLQ ráta:** alapzajként a szimulátor káosza látszik (~3%); ugrás = valami tényleg elromlott.
- **Feldolgozási p95:** ha nő, a worker/Redis szűk keresztmetszet.
- **Csomagok státusz szerint:** az üzleti állapot egy pillantásra.

10. Hands-on gyakorlatok

A gyakorlatok feltételezik, hogy a stack fut (lokálisan a `docker compose up --build` után, vagy a klaszteren), és van `kubectl` hozzáférése a `kafka-demo` namespace-hez. Minden gyakorlat önálló — bármelyik kihagyható vagy ismételhető.

10.1. Ismerkedés: kövess végig egy csomagot!

- Nyisd meg a dashboardot (lokálisan `http://localhost:8000`, klaszteren `https://kafka-demo.mattlab.hu`).
- Adj fel egy csomagot az űrlapon, és figyeld az élő folyamatban a FELVETEL eseményt. Kattints az azonosítóra: nézd meg az idővonalát.
- Ugyanezt API-ból: `curl -X POST .../api/parcels -d '{...}'` — a válasz 202, és a `parcel_id`-vel lekérdezhető.
- Nézd meg a `/docs` oldalt is: a FastAPI automatikus OpenAPI dokumentációja.

10.2. Consumer group skálázás és rebalance

```
kubectl -n kafka-demo logs -l app=kafka-demo-worker -f | grep Rebalance &
kubectl -n kafka-demo scale deploy kafka-demo-worker --replicas=4
# várj ~10 mp-et, figyeld a partíció-újraosztást, majd:
kubectl -n kafka-demo scale deploy kafka-demo-worker --replicas=1
```

Skálázás fel és le

- Hány partíciót kap egy-egy worker 4 replikánál? És 1-nél?
- Skálázz 8 replikára: mit csinál a 7. és 8. worker? (Tipp: 6 partíció van.)
- Eközben a dashboard élő folyama zavartalan? Miért?

10.3. DLQ: küldj poison pillt!

```
PID=$(curl -s -X POST https://kafka-demo.mattlab.hu/api/parcels \
-H 'Content-Type: application/json' \
-d '{"destination":"Szeged","weight_g":500}' | jq -r .parcel_id)
# azonnal KEZBESITVE-t küldünk — ez sorrendtörés:
curl -s -X POST "https://kafka-demo.mattlab.hu/api/parcels/$PID/events" \
-H 'Content-Type: application/json' \
-d '{"event_type":"KEZBESITVE","location":"Csalás"}'
```

Érvénytelen állapotátmenet provokálása

- Figyeld a worker logját: megjelenik az „Érvénytelen átmenet... → DLQ” sor.
- Olvasd ki a DLQ-t a Kafka CLI-vel (lásd a 12. fejezet parancspuskáját) — találd meg az üzenetedet, benne a hiba okával.
- A dashboard DLQ számlálója nőtt. A csomag státusza változott? Miért nem?

10.4. Idempotencia bizonyítása

- Nézd meg a worker logban egy tetszőleges esemény `event_id`-jét, majd Redis-ben: `kubectl -n kafka-demo exec deploy/kafka-demo-redis -- redis-cli KEYS 'dedup:*` (pár darabot listázz).
- Öld meg az egyik worker podot feldolgozás közben: `kubectl -n kafka-demo delete pod` — a K8s újat indít.

- A commit nélkül maradt köteg újrajátszódik. Keresd meg a logban a „Duplikált esemény kiszűrve” sorokat: ez az idempotencia működés közben.

10.5. A materializált nézet újraépítése (replay)

```
# 1. Üritsd ki a Redist (a nézet elveszik – a Kafka napló NEM!):
kubectl -n kafka-demo exec deploy/kafka-demo-redis -- redis-cli FLUSHALL
# 2. Állítsd le a workereket, reseteld az offsetet a topic elejére:
kubectl -n kafka-demo scale deploy kafka-demo-worker --replicas=0
kubectl -n kafka-demo exec kafka-demo-kafka-0 -- \
  /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group parcel-worker --reset-offsets --to-earliest \
  --topic parcel.events --execute
# 3. Indítsd vissza a workereket:
kubectl -n kafka-demo scale deploy kafka-demo-worker --replicas=2
```

Esemény-visszajátszás a gyakorlatban

- Figyeld a dashboardot: a számlálók nulláról másodpercek alatt visszaépülnek.
- Ez az event sourcing szupererő: az állapot eldobható, mert a napló a forrás.
- Kérdés: a „ma kézbesítve” szám pontosan ugyanaz lesz, mint előtte? (Tipp: a dedup-kulcsokat is töröltük — mégis miért jó az eredmény?)

10.6. Cache-aside megfigyelése

- Frissítsd a dashboardot gyorsan többször: a „Statisztika cache” kártya HIT és MISS között váltakozik (5 mp TTL).
- API-ból: `curl .../api/stats/overview | jq .cache` — mérd meg mindkét út válaszütemét `time-mal`.
- Gondolkodó: mi történne, ha a TTL 0 lenne? És ha 1 óra?

10.7. Self-healing és GitOps-drift

- Töröld kézzel az API Service-t: `kubectl -n kafka-demo delete svc kafka-demo-api` — majd nézd az ArgoCD felületét: az app `OutOfSync` lesz, és a `selfHeal` másodperceken belül visszaállítja.
- Skálázd kézzel a workert 5-re `kubectl scale -l`, és figyeld, ahogy az ArgoCD visszaigazítja a Gitben deklarált 2-re. A kézi `kubectl` a GitOps-világban csak kísérletezésre való — a valódi változás a Gitben történik.

10.8. Változtatás GitOps-módra

- Emeld a szimulátor forgalmát: a Kafka repóban a `helm/parcel-tracker/values.yaml`-ban `parcelsPerMinute: 10 → 30`, commit + push a main-re.
- Kövesd végig: GitHub Actions fut → ArgoCD Synced → a dashboard forgalma megugrik. Egyetlen `kubectl` parancs nélkül deployoltál.
- Bónusz: nézd meg a CI futásban a „values image tag frissítése” lépést — látni fogod a GitOps-visszaírást élőben.

10.9. Grafana: olvasd a rendszert!

- Nyisd meg a Grafanában a „Kafka Demo — Posta Csomagkövető” dashboardot (automatikusan megjelent — miért? Tipp: sidecar + ConfigMap label).

- A 10.2-es skálázási gyakorlat közben figyeld a „Feldolgozott események” panelt: látszik-e a rebalance okozta rövid megtorpanás?
- A 10.3-as DLQ-gyakorlat után keresd meg az ugrást a „DLQ ok szerint” panelen.

11. Merre tovább? A demótól az éles rendszerig

A demó tudatos egyszerűsítéseket tartalmaz — ez a fejezet arról szól, mi változna éles környezetben. Jó interjúkérdés-gyűjtemény is egyben.

Terület	A demóban	Élesben
Kafka topológia	1 broker, RF=1 (KRaft, combined mode)	3+ broker, RF=3, min.insync.replicas=2, külön controller
Kafka üzemeltetés	kézzel írt StatefulSet	Strimzi operátor: brokerek, useerek, topicok CRD-ként
Sémakezelés	JSON, közös pydantic modell	Schema Registry (Avro/Protobuf), kompatibilitási szabályok
Biztonság	PLAINTEXT a klaszteren belül	TLS + SASL auth, ACL-ek, NetworkPolicy, sealed secrets
Redis	1 példány, perzisztencia nélkül	Sentinel/Cluster vagy menedzselt szolgáltatás; AOF ha elsődleges tár
Skálázás	kézi replicas érték	HPA (CPU/memória) vagy KEDA (Kafka lag alapján!)
DLQ kezelés	megfigyeljük	riasztás + újrafeldolgozó (replay) folyamat + hibataxonómia
Megfigyelhetőség	metrikák + logok	+ elosztott trace-ek (OpenTelemetry), kafka-exporter, log-aggregáció
Kiadás	auto-sync a main-ről	környezetenkénti promóció, PR-alapú jóváhagyás, progressive delivery

TIPP — A KEDA + Kafka lag alapú skálázás különösen elegáns folytatás: a worker replikaszámát a tényleges feldolgozási lemaradás vezérli — nagy forgalomnál felskáláz, éjszaka nullára ereszt.

Ajánlott sorrend a mélyebb tanuláshoz: (1) Strimzi kipróbálása egy teszt namespace-ben, (2) séma-evolúció végiggondolása a ParcelEvent modellen (mi történik, ha új mező jön?), (3) KEDA bekötése a demó workerére, (4) NetworkPolicy írása a kafka-demo namespace-re.

12. Függelék: parancspuska

Kafka CLI (a broker podban futtatva)

```
ALIAS="kubectl -n kafka-demo exec kafka-demo-kafka-0 --"

# Topicok listája / részletei (partíciók, offsetek)
$ALIAS /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
$ALIAS /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --describe --topic parcel.events

# Consumer group állapot: partíció-kiosztás és LAG
$ALIAS /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group parcel-worker

# Üzenetek olvasása (fő topic / DLQ)
$ALIAS /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic parcel.events.dlq --from-beginning --max-messages 5
```

Redis CLI

```
R="kubectl -n kafka-demo exec deploy/kafka-demo-redis -- redis-cli"

$R HGETALL parcel:PT0123456789HU      # egy csomag állapota
$R LRANGE parcel:PT0123456789HU:history 0 -1  # idővonal
$R HGETALL stats:status_counts        # státusz-megoszlás
$R ZREVRANGE stats:center:events 0 9 WITHSCORES  # top központok
$R TTL cache:stats:overview           # cache hátralévő élettartama
$R INFO memory                        # memóriahasználat
$R MONITOR                            # ÉLŐ parancsfolyam (Ctrl+C!)
```

Kubernetes / Helm / ArgoCD

```
kubectl -n kafka-demo get pods -w      # podok élőben
kubectl -n kafka-demo logs -l app=kafka-demo-worker -f --tail=50
kubectl -n kafka-demo describe pod <pod>  # események, probe-ok
kubectl -n kafka-demo get events --sort-by=.lastTimestamp | tail
kubectl -n kafka-demo top pods           # erőforrás-fogyasztás

helm template kafka-demo helm/parcel-tracker | less
helm get values kafka-demo -n kafka-demo

argocd app get kafka-demo                # CLI-ből (opcionális)
argocd app history kafka-demo
```

Az API gyorsesztyje

```
BASE=https://kafka-demo.mattlab.hu      # lokálisan: http://localhost:8000

curl -s $BASE/healthz ; curl -s $BASE/readyz
curl -s -X POST $BASE/api/parcels -H 'Content-Type: application/json' \
  -d '{"destination":"Szeged","weight_g":1500,"sender":"Oktatás"}'
curl -s $BASE/api/parcels/<PARCEL_ID> | jq
curl -s $BASE/api/stats/overview | jq
curl -sN $BASE/api/live                  # SSE folyam (Ctrl+C)
```

Jó tanulást és jó demózást! Ha elakadsz: először a logok, aztán a metrikák, végül a Git-történet — ebben a sorrendben.